Agentic AI for Accounts Payable

Invoice three-way match with a team of cooperating agents

Authors: Swastik Gavhane, Saafalya Sengupta

PRN: 24070126224, 24070126158

Institution: Symbiosis Institute of Technology

Domain: Banking & Finance, Business Operations

Source code: https://github.com/Saafalya/invoice_agents

1. Abstract

Before paying a vendor, finance teams must confirm that the invoice agrees with the purchase order (what was ordered) and the goods receipt (what arrived). Done by hand this is slow and repetitive, and every miss becomes an overpayment or a duplicate payment. This project builds a team of five cooperating AI agents that check each invoice against its purchase order and goods receipt before payment, and that attach a written reason to every decision.

The system was validated on 24 synthetic invoices containing 8 seeded errors. It caught all 8 errors, identified every issue type correctly, wrongly flagged none of the 16 clean invoices, and flagged INR 1.77 lakh at risk, which equals the seeded total. These results come from rule-based matching on known data and should be read as proof that the pipeline works, not as a measure of real-world accuracy.

2. Problem Statement

Three-way match requires agreement between three documents:

Document	What it records	Question it answers
Purchase order (PO)	What we agreed to buy	What did we order?
Goods receipt (GRN)	What actually arrived	What did we receive?
Invoice	What the vendor bills	What are we being asked to pay?

All three must agree before payment is released. Errors that commonly slip through include:

- **Price above the PO:** the invoice charges a price above the agreed PO price;
- **Billed more than received:** the vendor bills for more units than the goods receipt shows;
- **Duplicate payment:** the same invoice is submitted and paid more than once.

3. From Manual Checks to Agents

Today: manual review	With agents
Staff compare three documents by hand	Planner orders the batch by urgency
Slow and repetitive for every invoice	Agents extract, validate and match
Errors often found after payment	Problems caught before payment
No consistent written reason	Every decision comes with a reason

Humans are not removed from the process: invoices that the agents flag as risky are still reviewed by a person.

4. System Design

4.1 The five agents

Each agent has a single responsibility.

Agent	Responsibility
Planner	Sequences the batch by due date and amount and flags urgent invoices.
Extraction	Turns invoice text into a typed schema using an LLM or a parser; retries if the result is rejected.
Critic	Rejects extractions that fail arithmetic checks and confirms that every variance is explained.
Matching	Compares invoice, PO and goods receipt on price, quantity, tax, vendor and duplicates.
Decision	Applies business rules to approve, hold or escalate, and writes the reason.

4.2 Agent hand-off and retry loop

The agents run in this order: Planner, Extraction, Critic (validate), Matching (three-way match), Critic (cross-check), Decision. The Critic therefore appears twice in the workflow.

The retry loop is the key agentic behaviour. If the Critic rejects an extraction, the Extraction Agent retries with a more robust parser. If it still fails, the invoice is escalated.

4.3 Decision outcomes

Outcome	When it applies
Approved	No issues. Safe to pay.
Hold	Low-value mismatch. The vendor is asked to correct it.
Escalated	Duplicate invoice, unknown vendor, or INR 10,000 or more at risk. A human reviews.

4.4 Architecture

The system is organised in layers:

- **User interface:** controls, a live agent trace and a results dashboard (Streamlit).
- **Orchestration:** a state graph with shared state, a conditional retry edge and a plain-Python fallback (LangGraph).
- **Agents:** Planner, Extraction, Critic, Matching and Decision.
- **Tools and data:** PO database, GRN database, vendor master, duplicate registry and business rules.
- **LLM:** an Ollama local model, or a mock model.

Data flows as follows: a batch enters the UI, the graph passes shared state between agents, the agents call tools and data, and the decision returns to the UI. Decision-making is rule-based (in tools.decide); the LLM is used only to write the explanation.

5. Demo Walkthrough

1. Generate batch: 24 synthetic invoices, 8 of them with seeded errors.
2. Run agents: watch each agent step live.
3. Review decisions: approved, held or escalated, colour-coded.
4. Inspect: raw invoice, agent trace, reason and metrics.

Live trace for one invoice (real output):

```
Extraction Agent: attempt 1 via fast parser: INV-7015 | total 34.00
Critic Agent: extraction REJECTED (qty x unit != amount) -> retry
Extraction Agent: attempt 2 via robust parser: INV-7015 | total 34,673.17
Matching Agent: 3-way match done -> 1 issue(s): PRICE_MISMATCH
Decision Agent: HOLD (low-value mismatch: return to vendor for correction)
```

The trace shows the Critic rejecting a misread total (34.00) and the retry recovering the correct total (34,673.17) before matching took place.

6. Results

Measured on 24 synthetic invoices: 8 with seeded errors and 16 clean.

Result	Score	Meaning
Seeded errors caught	8 / 8	Every seeded error was detected.
Issue types identified	8 / 8	Each error was classified with the correct issue type.
Clean invoices wrongly flagged	0 / 16	No false positives on clean invoices.
Amount at risk flagged	INR 1.77 lakh	Equal to the seeded total.

The Critic loop caught misread amounts, and the retry fixed them before matching. These high scores reflect rule-based matching on known data, not real-world accuracy: the answer key is known because the data is synthetic and the matching rules are deterministic.

7. Limitations and Future Scope

Limitations	Future scope
Synthetic data and one fixed invoice layout	OCR for scanned PDFs and photos
Rule-based matching scores optimistically	ERP connectors such as SAP and Tally
No OCR, ERP link or user login	Benchmark LLM extraction against the parser
Time savings are estimates, not measurements	Vendor fraud pattern detection
	Human approval screen and MCP tools

The biggest next step is moving to real invoices, using OCR together with an LLM extractor.

8. Conclusion

A small team of single-purpose agents, coordinated by a state graph and checked by a Critic, can run an invoice three-way match end to end, retry its own extraction errors, and explain every approve, hold or escalate decision. The prototype performs perfectly on its synthetic test set, and the honest next step is to test it on messy real-world invoices.

9. Project Links

GitHub repository: https://github.com/Saafalya/invoice_agents

Clone URL: `https://github.com/Saafalya/invoice_agents.git`